

 Universidade do Porto Faculdade de Engenharia FEUP	Faculdade de Engenharia da Universidade do Porto Mestrado Integrado em Engenharia Informática e Computação Programação em Lógica <i>Época Normal – Com Consulta / Duração: 2h30m</i>	2006/2007 3º Ano MIEIC
Nome:		Data: 16/01/2007

Nota: Utilize exclusivamente Prolog Standard e não utilize nenhuma biblioteca específica a não ser a clpfd no grupo 5. Responda aos grupos I e II em folhas separadas.

GRUPO I – Programação em Prolog

1) Ligas de Futebol (3.0 Val.)

Suponha a seguinte base de conhecimento em Prolog sobre as ligas, equipas e jogadores do campeonato português de futebol.

Exemplo da Base de Factos:

```

lig(super, porto).          jog(porto, quaresma, av).
lig(super, benfica).        jog(porto, helton, gr).
lig(super, sporting).       jog(benfica, simao, av).
lig(honra, feirense).       jog(sporting, moutinho, me).
lig(honra, guimaraes).      jog(sporting, ricardo, gr).
                           jog(guimaraes, cleber, de).
                           jog(feirense, edgar, me).

```

1.1) Escreva um predicado em Prolog *jogadores(+Posicao, -Lista)*, que retorne a Lista de todos os jogadores e respectivos clubes, que jogam numa dada posição. **(1.0 Val.)**.

Exemplo:

```

?- jogadores(av, Lista).
Lista = [quaresma-porto, simão-benfica].
?- jogadores(gr, Lista).
Lista = [helton-porto, ricardo-sporting].

```

1.2) Escreva um predicado em Prolog *calcula(-Lista)* que sobre esta base de factos com todos os jogadores de todas as equipas de todas as ligas de futebol português, dê uma lista cujos elementos são da forma [Ligai-[Eqj-[Jogk,...],...],...]. **(2.0 Val.)**.

Exemplo:

```

?- calcula(Lista).
Lista = [super-[porto-[quaresma, helton], benfica-[simao], sporting-
[moutinho, ricardo]], honra-[feirense-[edgar], guimarães-[cleber]]].

```

2) Celebridade na Festa (2.5 Val.)

Numa festa, uma pessoa é a celebridade da festa se for a única pessoa que toda a gente conhece. Por exemplo, na figura, está representada uma festa em que a celebridade é o Ivo (pois todos conhecem o Ivo!). Uma seta orientada de A para B significa que A conhece B (não o oposto). Dado que festas sem celebridades não são interessantes, nem sequer são consideradas. Festas com mais do que uma celebridade também não são interessantes. Pretende-se escrever um programa *celebridade(-Celeb)*, eficiente, para encontrar a celebridade da festa.

Exemplo de Problema:

```

pessoas([ana, ze, to, rui, ivo]).
conhece(ana, to). conhece(ana, ivo).
conhece(ana, ze). conhece(ze, ivo).
conhece(to, ivo). conhece(to, rui).
conhece(rui, to). conhece(rui, ivo).

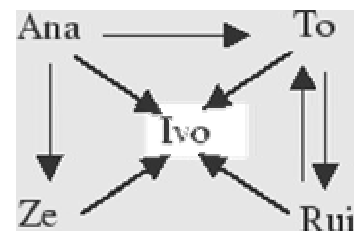
```

Exemplo:

```

?- celebridade(X).
X = ivo.

```



(problema adaptado de CeNPL 2003)

3) Questões Teórico-Práticas (3.0 Val.)

3.1) Diga quais as respostas do Prolog às seguintes questões, justificando e indicando o que é imprimido no écran. (2.0 Val.).

- a) ?- $5 + 6 = 6 + 5$.
- b) ?- $A = 2, B = 4, A + 6 =: 4 + B$.
- c) ?- $N \text{ is } 10, N \text{ is } N+1$.
- d) ?- $N = M, N = 5, \text{write}(M)$.
- e) ?- $\text{functor}(\text{Term}, f, 2), \text{arg}(2, \text{Term}, 4), \text{Term} =.. [A,2,B]$.
- f) ?- $[a1,a2|a3] =.. [C,D,E]$.

3.2) Qual a diferença entre usar apropriadamente uma “guarda” ou um “cut” numa cláusula que queremos provar? Exemplifique. (1.0 Val.).

4) Existência de Padrões em Strings (4.0 Val.)

Implemente um predicado Prolog para verificar se um padrão existe numa string. O predicado recebe como primeiro argumento a string e como segundo o padrão. O predicado sucede se o padrão existir na string e falha no caso contrário. Considere que a string só contém letras minúsculas. Considere ainda que o padrão é descrito por uma expressão regular que pode conter unicamente letras minúsculas e

4.1) O símbolo '+' que representa uma qualquer letra (2.0 Val.).

Exemplo:

```
?- existePadrao("aaabfa", "bf+").
yes.
?- existePadrao("aabf", "bf+").
no.
?- existePadrao("aabf", "a+b").
yes.
```

4.2) O símbolo '*' (com o mesmo significado) e o símbolo '*' que representa qualquer sequência de zero ou mais letras. (2.0 Val.).

Exemplo:

```
?- existePadrao("caaabfvv", "ab*f+").
yes.
?- existePadrao("aabfg", "b+*fg").
no.
?- existePadrao("caaabfvv", "*").
yes.
?- existePadrao("caaabfvv", "c*++v").
yes.
```

Sugestão: Utilize inicialmente o predicado: *string_to_list(?String, ?List)*. Por exemplo:

```
?- string_to_list("a*bxyz", X).
X = [97, 43, 42, 98, 120, 121, 122].
```

GRUPO II – Programação em Lógica com Restrições

5.1) Construa um programa *solveDigitos(-A,-B)*, utilizando PLR para encontrar os dois números (A e B) de três dígitos que obedecem às seguintes condições: **(2.0 Val.)**

- Os seis dígitos que compõem os números A e B são todos distintos e nenhum deles é o 0.
- O primeiro dígito de B é igual a metade do último dígito de A.
- Em ambos os números os dígitos encontram-se ordenados por ordem crescente.
- A soma dos dígitos de A é igual à soma dos dígitos de B.
- A multiplicação dos dígitos de A adicionado a 12 é igual à multiplicação dos dígitos de B.

Notas: Não esquecer de utilizar a biblioteca clpfd e que as variáveis utilizadas são inteiras (domínio finito).

Exemplo:

```
?- solveDigitos(A,B).  
A = 378, B = 459.
```

5.2) Cinco amigos de nomes predestinados, *Coelho*, *Pato*, *Rola*, *Pombo* e *Lebre*, regressam de uma caçada. Trazem consigo cinco animais correspondentes aos respectivos cinco nomes: **(2.5 Val.)**

- Cada um matou um único animal, que não corresponde, porém, ao seu nome.
- Cada um falhou um animal diferente do que matou e que não corresponde também ao próprio nome.
- O *coelho* foi morto pelo caçador cujo nome é o do animal falhado por *Rola*.
- O *pato* foi morto pelo caçador cujo nome é o do animal falhado por *Lebre*.
- *Pato*, que falhou uma *lebre*, ficou muito aborrecido por ter morto apenas uma *rola*.

Quais os bichos mortos e falhados por cada um dos 5 caçadores? Escreva um programa utilizando PLR *jogo/0*, que resolva este puzzle, i.é, determine a resposta à questão acima colocada. O programa deve mostrar no ecrã os 5 triplos: *Cacador/Animal_Morto/Animal_Falhado*

(adaptado de "Pierre Berloquin, 100 Jogos Lógicos, O Prazer da Matemática, no 4, Gradiva")

5.3) Pretende-se calendarização um campeonato de futebol com *n* equipas que jogam em *n/2* campos distintos. As equipas disputam unicamente *m* jornadas (em que $m \leq n/2$). Em cada jornada todas as *n* equipas jogam um jogo. Cada equipa joga no máximo uma vez em cada campo e joga no máximo uma vez com cada oponente. Existem restrições rígidas para alguns jogos e restrições de precedência entre alguns jogos. O problema é representado da seguinte forma (exemplo simples): **(3.0 Val.)**

Exemplo de Problema:

```
jornadas(6).           //calendário com 6 jornadas  
equipas(12).           //12 equipas em 6 campos  
jogo(1-2, 1, 1).       //jogo entre equipas 1 e 2 na 1ª jornada no campo 1.  
jogo(5-9, 3, 2).  
jogo(4-5, 4, 5).  
jogo(2-6, 2, 5).  
precede(1-2, 3-5).     //jogo entre equipas 1 e 2 é numa jornada anterior  
precede(1-4 9-12).     ao jogo entre 3 e 5
```

O programa deve imprimir a lista de jogos formatada como no exemplo seguinte:

Exemplo:

```
?- solve(L).  
L = [1,2,3,4,5,6,7,8,9,10,11,12,3,5,8,11,10,12,1,4,2,6,7,9,  
4,6,5,9,2,7,10,11,8,12,1,3,7,11,1,12,3,8,2,9,4,5,6,10,  
8,10,6,7,1,9,5,12,3,11,2,4,9,12,2,10,4,11,3,6,1,7,5,8].
```

Solução:

Jorn-1:	1, 2,	3, 4,	5, 6,	7, 8,	9,10,	11,12,
Jorn-2:	3, 5,	8,11,	10,12,	1, 4,	2, 6,	7, 9,
Jorn-3:	4, 6,	5, 9,	2, 7,	10,11,	8,12,	1, 3,
Jorn-4:	7,11,	1,12,	3, 8,	2, 9,	4, 5,	6,10,
Jorn-5:	8,10,	6, 7,	1, 9,	5,12,	3,11,	2, 4,
Jorn-6:	9,12,	2,10,	4,11,	3, 6,	1, 7,	5, 8]